Tugas Kelompok 1

Technical Report

Analisis Numerik B

Anggota:

Farras Hafizhudin Indra Wijaya (2106652682)

Haekal Alexander Dinova (2406352424)

Anderson Tirza Liman (2406355893)

Jovanus Irwan Susanto (2406434140)

Pakta Integritas

Dengan ini, kami menyatakan bahwa tugas ini adalah hasil pekerjaan kelompok kami sendiri.

Kelompok B3

Farras Hafizhudin Indra Wijaya	Haekal Alexander Dinova	Anderson Tirza Liman	Jovanus Irwan Susanto

Daftar Isi

Technical Report I: Block LU dan Thomas Algorithm untuk Sistem Linear Banded **3**

1 Pendahuluan **3**

2 Karakteristik Matriks dan Rancangan Algoritma **4**

2.1 Karakteristik Matriks dan Optimisasi Penyimpanan 4

2.2 Rancangan Block LU 5

2.3 Rancangan Thomas Algorithm 7

3 Hasil Eksperimen **8**

3.1 Perbandingan Runtime dan Pemakaian Resource 8

4 Analisis Kompleksitas **9**

4.1 Block LU 9

4.2 Thomas Algorithm 10

4.3 Membaca Tren Running Time 11

5 Analisis Kondisi dan Error **11**

6 Kesimpulan dan Rekomendasi **12**

Referensi Technical Report I **13**

Technical Report II: Pemodelan Lintasan Bintang **14**

7 Pendahuluan **14**

8 Pembahasan **15**

8.1 Pembentukan Model Least Squares 15

8.2 Penyelesaian dengan Normal Equation 16

8.3 Penyelesaian dengan QR Householder 16

8.4 Perbandingan Kompleksitas FLOPs dan Memori 18

8.5 Evaluasi Stabilitas Numerik 18

8.6 Persamaan Elips Akhir dan Interpretasi Geometris 19

8.7 Validasi Visual 21

9 Kesimpulan **22**

Referensi Technical Report II **23**

Lampiran: Repositori dan Asal Berkas **23**

Technical Report I: Block LU dan Thomas Algorithm untuk Sistem Linear Banded

Rangkuman

Laporan pertama membahas implementasi dan evaluasi dua pendekatan untuk menyelesaikan sistem persamaan linear banded, yaitu **Block LU Factorization** dan **Thomas algorithm**. Dari identifikasi struktur matriks input, diperoleh bahwa data asli mempunyai lower bandwidth $p = 1$ dan upper bandwidth $q = 2$, sehingga matriks memang berada pada kelas *banded system*. Fakta ini penting karena pada kelas banded, pilihan algoritma seharusnya tidak lagi dipandang sebagai satu solusi tunggal untuk semua kasus, melainkan disesuaikan dengan struktur pita yang tersedia.

Dalam kerangka ini, Block LU dipakai sebagai metode *general-purpose* untuk *banded system* yang belum cukup spesial untuk ditangani oleh solver compact seperti Thomas atau variasinya. Implementasi yang dipakai pada laporan ini tetap menyimpan matriks kerja dalam bentuk dense, tetapi organisasi blocked-nya menjaga locality komputasi lebih baik daripada LU non-blocked. Kebijakan backend yang dipakai adalah: untuk ukuran $n < 512$, perhitungan dilakukan di CPU, sedangkan untuk $n \geq 512$, operasi *triangular solve* dan Schur complement dipindahkan ke GPU. Sebaliknya, Thomas algorithm dipakai untuk kasus tridiagonal, yaitu subkelas banded yang memang mempunyai struktur cukup spesifik untuk dieksploitasi langsung dengan array satu dimensi.

Hasil eksperimen menunjukkan bahwa pada matriks tridiagonal sintetis berukuran sampai 512, Thomas algorithm selalu lebih cepat daripada Block LU. Sebagai contoh, pada ukuran 512, Block LU membutuhkan sekitar 21.5605 ms, sedangkan Thomas algorithm hanya sekitar 5.3477 ms. Analisis kompleksitas mendukung temuan ini: Block LU tetap memiliki biaya waktu orde $O(n^3)$ dan memori orde $O(n^2)$, sedangkan Thomas algorithm hanya membutuhkan waktu dan memori orde $O(n)$. Dari sisi akurasi, error Thomas algorithm tetap sangat kecil dan stabil pada seluruh ukuran yang diuji.

Kesimpulan utamanya adalah bahwa pemilihan algoritma harus mengikuti struktur matriks. Untuk *banded system* yang tidak mempunyai struktur cukup spesial untuk solver khusus, Block LU masih merupakan pendekatan umum yang masuk akal. Namun untuk *banded system* spesial seperti matriks tridiagonal, algoritma yang sadar struktur seperti Thomas jauh lebih efisien tanpa mengorbankan akurasi.

1 Pendahuluan

Tujuan utama studi ini adalah membandingkan dua pendekatan penyelesaian sistem linear banded, yaitu Block LU dan Thomas algorithm, dengan memperhatikan tiga aspek utama: efisiensi penyimpanan, biaya komputasi, dan akurasi solusi. Motivasi studi ini muncul dari fakta bahwa matriks input sering kali mempunyai struktur pita yang bisa dikenali lebih awal. Bila struktur tersebut diketahui sejak awal, maka baik struktur data maupun algoritma penyelesaiannya seharusnya dapat dipilih secara lebih tepat.

Dalam laporan ini, pembahasan dimulai dari identifikasi karakteristik matriks dan konsekuensinya terhadap penyimpanan. Setelah itu, dua algoritma yang digunakan dipa-

parkan, dimulai dari Block LU sebagai metode umum untuk *banded system* yang tidak mempunyai solver spesifik yang sederhana, lalu dilanjutkan dengan Thomas algorithm sebagai metode khusus tridiagonal. Sesudah pembaca memiliki gambaran yang cukup tentang cara kerja algoritma, hasil eksperimen, analisis kompleksitas, serta analisis kondisi dan error dibahas secara bertahap.

2 Karakteristik Matriks dan Rancangan Algoritma

2.1 Karakteristik Matriks dan Optimisasi Penyimpanan

Untuk setiap elemen tak nol A_{ij}, lower bandwidth dan upper bandwidth didefinisikan sebagai

$$p = \max\{i - j \mid A_{ij} \neq 0\}, \qquad q = \max\{j - i \mid A_{ij} \neq 0\}.$$

Dengan definisi ini, struktur pita matriks dapat dikenali langsung dari file CSV. Pada keenam matriks asli berukuran $8, 16, 32, 128, 256, 512$, hasil identifikasinya konsisten: seluruh matriks memiliki $p = 1$ dan $q = 2$. Artinya, matriks input merupakan *banded matrix* yang sempit, sehingga pembahasan memang seharusnya berfokus pada keluarga *banded system*, bukan pada matriks penuh tanpa struktur.

```
procedure FIND_BANDWIDTH(A, tau)
    n <- size(A)
    p <- 0
    q <- 0

    for i = 1 to n:
        for j = 1 to n:
            if abs(A[i,j]) <= tau:
                continue
            if i >= j:
                p <- max(p, i-j)
            else:
                q <- max(q, j-i)

    return p, q
end procedure
```

Kode 2.1: Pseudocode pencarian lower bandwidth p dan upper bandwidth q.

Pada Kode 2.1, setiap elemen tak nol di bawah diagonal utama memberi kandidat nilai $i-j$ untuk p, sedangkan setiap elemen tak nol di atas diagonal utama memberi kandidat nilai $j-i$ untuk q. Nilai maksimum dari kedua selisih tersebut adalah bandwidth matriks.

Tabel 2.1 memperlihatkan mengapa penyimpanan dense segera menjadi boros. Untuk ukuran 512×512, hanya 2044 elemen yang tak nol, tetapi representasi dense tetap memaksa penyimpanan 262144 elemen. Secara teoritis, representasi dense membutuhkan $O(n^2)$ ruang, sedangkan representasi banded hanya membutuhkan $O((p+q+1)n)$. Karena pada data ini p dan q konstan, kebutuhan ruangnya efektif menjadi $O(n)$.

Untuk data asli dengan $p = 1$ dan $q = 2$, struktur compact yang paling alami adalah empat diagonal 1D: satu subdiagonal, satu diagonal utama, dan dua superdiagonal. Dalam

Table 2.1: Perbandingan penyimpanan dense dengan jumlah elemen tak nol pada data asli.

N	Dense n^2	Nonzero aktual	Rasio dense / nonzero
8	64	28	2.29×
16	256	60	4.27×
32	1024	124	8.26×
128	16384	508	32.25×
256	65536	1020	64.25×
512	262144	2044	128.25×

eksperimen lanjutan juga dibangun kasus tridiagonal; pada kasus inilah Thomas algorithm menjadi relevan karena seluruh sistem dapat direpresentasikan dengan sejumlah kecil array 1D.

2.2 Rancangan Block LU

Block LU dipilih sebagai metode umum untuk *banded system* yang tidak mempunyai struktur cukup spesial untuk solver compact seperti Thomas. Dalam implementasi laporan ini, matriks kerja memang masih disimpan dalam bentuk dense, tetapi bentuk blocked tetap lebih sesuai dengan perilaku mesin aktual daripada LU non-blocked yang bekerja terlalu halus pada elemen-elemen kecil. Pada Block LU, panel diagonal kecil cenderung tetap hangat di cache, sedangkan update mahal pada trailing matrix dipadatkan menjadi operasi blok besar. Konsekuensinya, reuse data membaik dan backend numerik dapat bekerja lebih efisien.

Secara konseptual, matriks $A \in \mathbb{R}^{N \times N}$ dibagi menjadi

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

dengan

$$A_{11} \in \mathbb{R}^{b \times b}, \qquad A_{12} \in \mathbb{R}^{b \times (N-b)}, \qquad A_{21} \in \mathbb{R}^{(N-b) \times b}, \qquad A_{22} \in \mathbb{R}^{(N-b) \times (N-b)}.$$

Di sini b adalah *block size*. Setelah blok diagonal A_{11} difaktorkan menjadi $L_{11}U_{11}$, blok kanan atas dan kiri bawah diselesaikan dengan *triangular solve*, lalu trailing matrix A_{22} diperbarui melalui Schur complement.

```
procedure PANEL_LU_INPLACE(A)
    n <- size(A)
    if n <= 1:
        return

    pivot <- A[1,1]
    m <- A[2:n,1] / pivot
    A[2:n,1] <- m

    A[2:n,2:n] <- A[2:n,2:n] - m outer A[1,2:n]
    PANEL_LU_INPLACE(A[2:n,2:n])
end procedure
```

Kode 2.2: Pseudocode panel LU in-place pada blok diagonal kecil.

Dalam implementasi aktual, bentuk pada Kode 2.2 direalisasikan sebagai in-place Gaussian elimination iteratif pada panel diagonal. Baris yang paling penting adalah

$$A_{2:n,\,2:n} \leftarrow A_{2:n,\,2:n} - m \otimes A_{1,\,2:n}.$$

Jika $r = A_{1,\,2:n}$ adalah sisa baris pivot dan $m = A_{2:n,\,1}/A_{1,1}$ adalah vektor multiplier, maka $m \otimes r$ membentuk matriks yang baris ke-i-nya sama dengan $m_i r$. Karena itu, satu baris pivot digunakan untuk mengupdate seluruh baris sisanya sekaligus.

```
procedure BLOCK_LU(A, b)
    n <- size(A)
    if n <= b:
        PANEL_LU_INPLACE(A)
        return A

    partition A into:
        [ A11  A12 ]
        [ A21  A22 ]
    with A11 of size b x b
         A12 of size b x (n-b)
         A21 of size (n-b) x b
         A22 of size (n-b) x (n-b)

    PANEL_LU_INPLACE(A11)
    define A11 = L11 * U11
    with L11 unit lower triangular
         U11 upper triangular

    U12 <- solve_lower_triangular(L11, A12, unit_diagonal = true)
    A12 <- U12
    L21 <- transpose(
             solve_lower_triangular(transpose(U11),
                                    transpose(A21),
                                    unit_diagonal = false)
            )
    A21 <- L21
```

```
    A22 <- A22 - L21 * U12

    BLOCK_LU(A22, b)
    return A
end procedure
```

Kode 2.3: Pseudocode Block LU utama.

Seperti halnya Kode 2.2, Kode 2.3 di atas akan diimplementasikan secara iteratif.

Secara konseptual, Block LU tetap berjalan melalui empat fase utama berikut.

1. Faktorisasi panel diagonal kecil dengan in-place Gaussian elimination.
2. Penyelesaian blok kanan atas U_{12} dari sistem $L_{11}U_{12} = A_{12}$.
3. Penyelesaian blok kiri bawah L_{21} dari sistem $L_{21}U_{11} = A_{21}$.
4. Pembentukan Schur complement $A_{22} \leftarrow A_{22} - L_{21}U_{12}$.

Schur complement adalah update trailing matrix

$$L_{21}U_{12} = \sum_{t=1}^{b} L_{21}(:,t)\, U_{12}(t,:),$$

yakni penjumlahan b buah *outer product*; dengan kata lain, ia adalah versi blok dari update outer product pada in-place Gaussian elimination.

Ukuran blok tidak diturunkan dari satu rumus universal, tetapi dipilih secara **heuristic**. Pendekatan ini sejalan dengan praktik LAPACK, khususnya fungsi `ILAENV`, yang memang mengembalikan parameter blok optimal secara *machine-dependent* (LAPACK Users' Guide, n.d.). Rancangan blocked LU seperti ini juga konsisten dengan implementasi `DGETRF` pada LAPACK, yang memisahkan pekerjaan panel dan update trailing matrix menjadi operasi blok (LAPACK DGETRF Source, n.d.). Karena itu, implementasi ini memakai block size kecil pada CPU dan block size lebih besar pada GPU.

Penyimpanan faktor dilakukan secara **in-place**. Karena diagonal matriks L selalu bernilai satu, diagonal tersebut tidak perlu disimpan eksplisit. Akibatnya, bagian bawah diagonal utama dari salinan A dipakai untuk multiplier L, sedangkan diagonal utama dan bagian atasnya dipakai untuk U.

Untuk ukuran `n < 512`, seluruh jalur Block LU dikerjakan di CPU. Untuk ukuran `n >= 512`, panel diagonal kecil tetap difaktorkan di CPU, tetapi *solve lower triangular*, *solve upper triangular*, dan pembentukan Schur complement dipindahkan ke GPU.

2.3 Rancangan Thomas Algorithm

Untuk matriks tridiagonal digunakan **Thomas algorithm**, yaitu bentuk khusus eliminasi Gauss untuk pita tiga diagonal. Algoritma ini menggunakan lima array 1D: subdiagonal a, diagonal utama b, superdiagonal c, ruas kanan d, dan solusi x. Dalam bentuk faktorisasi eksplisit, array a dapat dioverwrite menjadi multiplier l, sedangkan array b dioverwrite menjadi diagonal baru u.

```
procedure THOMAS(a, b, c, d)
    for i = 1 to n-1:
        l[i] <- a[i] / b[i]
        b[i+1] <- b[i+1] - l[i] * c[i]
        d[i+1] <- d[i+1] - l[i] * d[i]

    x[n] <- d[n] / b[n]
    for i = n-1 down to 1:
        x[i] <- (d[i] - c[i] * x[i+1]) / b[i]

    return x
end procedure
```

Kode 2.4: Pseudocode Thomas algorithm.

Berbeda dari Block LU yang ditujukan sebagai metode umum untuk *banded system* non-spesial dan pada implementasi ini masih bekerja dengan matriks kerja dense, Thomas langsung memanfaatkan struktur pita tiga diagonal. Karena itu, ia bukan sekadar versi yang lebih ringan, melainkan memang algoritma yang dirancang untuk subkelas matriks yang lebih spesifik.

3 Hasil Eksperimen

3.1 Perbandingan Runtime dan Pemakaian Resource

Perbandingan eksperimen dibatasi pada ukuran sampai 512. Kasus yang dipakai adalah matriks tridiagonal sintetis yang konsisten dengan Thomas algorithm. Pada Block LU diberlakukan kebijakan backend yang sama seperti implementasi utama: **CPU** untuk $n < 512$ dan **CPU+GPU** untuk $n \geq 512$.

Table 3.1: Perbandingan runtime dan peak memory inti algoritma antara Block LU dan Thomas algorithm.

N	Backend BLU	BLU time (ms)	Thomas time (ms)	BLU algo mem (MB)	Thomas algo mem (MB)
8	CPU	0.2944	0.0756	0.0022	0.0006
16	CPU	0.6092	0.1500	0.0083	0.0012
32	CPU	1.3113	0.2959	0.0322	0.0024
128	CPU	4.8423	1.1687	0.5039	0.0097
256	CPU	11.2256	2.3781	2.0078	0.0195
512	CPU+GPU	21.5605	5.3477	8.0078	0.0390

Tabel 3.1 dan Tabel 3.2 menunjukkan pola yang tegas. Thomas selalu lebih cepat karena ia memang ditujukan untuk struktur tridiagonal compact, sedangkan Block LU di sini berperan sebagai metode yang lebih umum untuk *banded system* dan masih membawa overhead representasi dense pada matriks kerja. Kolom memory pada Tabel 3.1 dihitung sebagai *peak core memory* yang dilacak dari array-array aktual yang masih hidup selama factorization dan solve, dengan view yang berbagi buffer tidak dihitung ganda sehingga angka yang dilaporkan merepresentasikan okupasi memori inti algoritma. Angka utilisasi

CPU dan GPU tetap diperoleh dari *batched profiling*, bukan dari satu eksekusi tunggal, agar durasi pengukuran cukup panjang untuk sampler.

Table 3.2: Perbandingan CPU usage dan GPU usage antara Block LU dan Thomas algorithm.

N	Backend BLU	BLU CPU avg (%)	Thomas CPU avg (%)	BLU GPU peak (%)	Thomas GPU peak (%)
8	CPU	6.39	5.67	0	0
16	CPU	6.15	6.86	0	0
32	CPU	6.32	6.23	0	0
128	CPU	5.66	5.70	0	0
256	CPU	5.26	5.08	0	0
512	CPU+GPU	4.49	5.11	36	0

4 Analisis Kompleksitas

4.1 Block LU

Total FLOPs dan Time Complexity

Untuk membuat penghitungan FLOPS lebih transparan, misalkan $n = sb$ dengan s banyaknya blok dan b adalah block size. Pada iterasi blok ke-j, definisikan

$$m_j = n - (j+1)b,$$

yaitu ukuran sisi trailing matrix setelah panel ke-j selesai diproses.

Table 4.1: Akumulasi FLOPS per fase pada Block LU.

Fase	FLOPS per iterasi ke-j	Akumulasi sampai selesai
Panel LU pada A_{11}	$\frac{b(b-1)(4b+1)}{6}$	$\frac{n(b-1)(4b+1)}{6}$
Hitung U_{12}	$m_j\, b(b-1)$	$\frac{n(n-b)(b-1)}{2}$
Hitung L_{21}	$m_j\, b^2$	$\frac{n(n-b)b}{2}$
Schur complement $A_{22} \leftarrow A_{22} - L_{21}U_{12}$	$2m_j^2 b$	$\frac{n(n-b)(2n-b)}{3}$
Total	-	$\frac{n(b-1)(4b+1)}{6} + \frac{n(n-b)(2b-1)}{2} + \frac{n(n-b)(2n-b)}{3}$
Asimtotik	-	$O(n^3)$

Tabel 4.1 memperlihatkan bahwa biaya dominan Block LU datang dari pembentukan Schur complement. Secara asimtotik, Block LU mempunyai kompleksitas waktu

$$T(n) = O(n^3).$$

Leading-order flop count faktorisasinya tetap sekitar

$$\frac{2}{3}n^3,$$

dengan tambahan sekitar $2n^2$ flop untuk dua kali *forward-back substitution* pada dua ruas kanan. Jadi, untuk satu faktorisasi dan dua solusi, total kerja aritmetiknya didominasi oleh

$$\frac{2}{3}n^3 + 2n^2.$$

Memory Complexity

Jika satu *unit memory* dipahami sebagai satu bilangan floating-point, maka okupasi utama Block LU dapat diringkas sebagai berikut.

Table 4.2: Ocupancy memori per struktur data pada Block LU.

Struktur data	Okupasi (unit memory)
Matriks kerja gabungan A/LU	n^2
Blok sementara L_{11}	b^2
Blok sementara U_{11}	b^2
Blok sementara U_{12}	$b(n-b)$
Blok sementara L_{21}	$(n-b)b$
Total peak CPU path	$n^2 + 2b^2 + 2b(n-b)$
Asimtotik	$O(n^2)$

Tabel 4.2 menunjukkan bahwa, walaupun ada beberapa blok sementara, seluruh okupasi tetap didominasi oleh satu matriks kerja dense besar berukuran n^2. Inilah biaya yang harus dibayar ketika Block LU dipakai sebagai pendekatan umum, bukan sebagai solver compact yang spesifik terhadap struktur pita tertentu.

4.2 Thomas Algorithm

Total FLOPs dan Time Complexity

Untuk Thomas algorithm, pemisahan per fase lebih sederhana karena algoritmanya memang hanya terdiri dari faktorisasi maju dan penyelesaian sistem segitiga. Karena pada eksperimen ini diselesaikan dua ruas kanan, maka fase solve dihitung dua kali.

Table 4.3: Akumulasi FLOPS per fase pada Thomas algorithm.

Fase	FLOPS	Keterangan
Faktorisasi maju	$3(n-1)$	1 divisi + 1 perkalian + 1 pengurangan per langkah
Solve untuk x_b	$5n-4$	forward substitution + backward substitution
Solve untuk x_c	$5n-4$	forward substitution + backward substitution
Total	$13n-11$	berasal dari $3(n-1)+(5n-4)+(5n-4)$
Asimtotik	$O(n)$	

Dari Tabel 4.3, kompleksitas waktunya langsung terlihat:

$$T(n) = O(n).$$

Perbedaan terhadap Block LU bukan hanya pada konstanta, tetapi juga pada kelas kompleksitas waktunya.

Memory Complexity

Thomas jauh lebih hemat karena tidak pernah membangun matriks dense penuh. Jika dihitung dalam satuan bilangan floating-point, okupasinya dapat dijelaskan per struktur data sebagai berikut.

Table 4.4: Occupancy memori per struktur data pada Thomas algorithm.

Struktur data	Okupasi (unit memory)
Subdiagonal / multipliers	$n-1$
Diagonal utama / u diagonal	n
Superdiagonal	$n-1$
Buffer RHS / solusi saat ini	n
Buffer kerja maju-mundur	n
Total peak implementasi	$5n-2$
Asimtotik	$O(n)$

Tabel 4.4 menunjukkan bahwa total okupasi Thomas tetap linear terhadap n. Jika implementasi dibuat lebih ketat dan beberapa buffer dioverwrite, konstanta ini masih bisa diperkecil, tetapi kesimpulan asimtotiknya tidak berubah.

4.3 Membaca Tren Running Time

Tren running time perlu dibaca bersama Tabel 3.1. Untuk Block LU, waktu komputasi naik dari 0.2944 ms pada ukuran 8 menjadi 21.5605 ms pada ukuran 512. Kenaikan ini jauh lebih tajam daripada linear dan konsisten dengan biaya komputasi metode umum yang dalam implementasi ini masih ditopang oleh matriks kerja dense dan orde kubik. Sebaliknya, Thomas naik dari 0.0756 ms menjadi 5.3477 ms pada rentang ukuran yang sama, yang jauh lebih landai dan selaras dengan kompleksitas linear.

Tabel eksperimen juga memperlihatkan detail implementasi. Sampai ukuran 256, Block LU masih berjalan di CPU dan waktu meningkat bertahap. Pada ukuran 512, backend berpindah ke CPU+GPU dan GPU mulai terlibat, tetapi ukuran masalah masih terlalu kecil untuk menutup overhead pendekatan umum yang masih memakai matriks kerja dense serta koordinasi device. Karena itu, Thomas tetap lebih cepat pada seluruh ukuran yang diuji.

5 Analisis Kondisi dan Error

Untuk *conditioning number*, digunakan tiga norma standar:

$$\kappa_1(A) = \|A\|_1\|A^{-1}\|_1, \qquad \kappa_2(A) = \|A\|_2\|A^{-1}\|_2, \qquad \kappa_\infty(A) = \|A\|_\infty\|A^{-1}\|_\infty.$$

Ketiga ukuran ini memisahkan dua hal yang sering bercampur dalam diskusi implementasi: performa algoritma dan sensitivitas numerik matriks itu sendiri.

Table 5.1: Condition number data asli berukuran 8 sampai 512.

N	$\kappa_1(A)$	$\kappa_2(A)$	$\kappa_\infty(A)$	Interpretasi
8	1.12×10^2	8.11×10^1	1.91×10^2	baik
16	7.10×10^1	3.02×10^1	7.88×10^1	baik
32	1.52×10^2	8.34×10^1	3.56×10^2	baik
128	1.06×10^6	4.06×10^5	2.48×10^6	mulai sensitif
256	1.13×10^{20}	3.18×10^{19}	3.38×10^{19}	ill-conditioned
512	7.84×10^4	2.65×10^4	1.01×10^5	moderat

Tabel 5.1 memperlihatkan bahwa sebagian besar sistem masih berada pada tingkat conditioning yang masuk akal, kecuali ukuran 256 yang jelas *ill-conditioned*. Untuk bagian error, laporan ini memakai Thomas algorithm, karena algoritma itulah yang secara struktural memang ditujukan untuk matriks tridiagonal yang dipakai pada eksperimen perbandingan hingga ukuran 512.

Table 5.2: Error Thomas algorithm pada matriks tridiagonal sampai ukuran 512.

N	LU err	x_b err	x_c err	r_b	r_c
8	1.85×10^{-17}	0	1.11×10^{-16}	0	4.87×10^{-17}
16	1.85×10^{-17}	1.11×10^{-16}	1.11×10^{-16}	4.93×10^{-17}	4.90×10^{-17}
32	1.85×10^{-17}	1.11×10^{-16}	1.11×10^{-16}	4.93×10^{-17}	4.92×10^{-17}
128	1.85×10^{-17}	1.11×10^{-16}	1.11×10^{-16}	4.93×10^{-17}	4.93×10^{-17}
256	1.85×10^{-17}	1.11×10^{-16}	1.11×10^{-16}	4.93×10^{-17}	4.93×10^{-17}
512	1.85×10^{-17}	1.11×10^{-16}	1.11×10^{-16}	4.93×10^{-17}	4.93×10^{-17}

Di sini r_b dan r_c menyatakan residual ternormalisasi untuk dua ruas kanan. Tabel 5.2 menunjukkan bahwa error Thomas tetap sangat kecil dan praktis stabil terhadap pertumbuhan ukuran hingga 512. Hal ini selaras dengan sifat masalah tridiagonal yang dipakai pada eksperimen: struktur matriks cocok dengan algoritmanya, sehingga solusi yang dihasilkan tetap akurat sekaligus murah secara komputasi.

Walaupun demikian, keterkaitan antara Tabel 5.1 dan Tabel 5.2 perlu dibaca dengan hati-hati. Pada kasus ini, nilai conditioning yang tinggi belum cukup menimbulkan akibat besar dari sisi error yang terukur. Error solusi dan residual tetap sangat kecil, sehingga secara praktis sistem yang diselesaikan masih berada pada rezim yang stabil. Namun tabel condition number tetap penting sebagai peringatan: jika sensitivitas matriks meningkat lebih jauh atau perturbasi data menjadi lebih besar, maka error solusi dapat mulai membesar secara jauh lebih nyata.

6 Kesimpulan dan Rekomendasi

Studi ini menunjukkan bahwa struktur matriks harus menjadi pertimbangan utama dalam memilih representasi data dan algoritma penyelesaian. Dari identifikasi band-

width pada data asli, terlihat bahwa matriks bersifat banded sempit dengan $p = 1$ dan $q = 2$, sehingga penyimpanan dense sebenarnya tidak ekonomis. Dalam konteks ini, Block LU sebaiknya dipahami sebagai metode umum untuk *banded system* yang belum mempunyai solver spesifik yang sederhana, bukan sebagai pilihan pertama untuk setiap struktur pita. Keunggulannya terletak pada organisasi blocked yang lebih sadar cache dibanding LU non-blocked dan pada kemampuannya memanfaatkan GPU pada operasi tertentu.

Eksperimen pada matriks tridiagonal sampai ukuran 512 memperlihatkan bahwa Thomas algorithm selalu lebih cepat daripada Block LU. Secara konkret, pada ukuran 512 Thomas membutuhkan sekitar 5.3477 ms, sedangkan Block LU membutuhkan sekitar 21.5605 ms. Analisis kompleksitas menjelaskan hasil ini secara langsung: Block LU tetap mempunyai biaya waktu orde $O(n^3)$ dan memori orde $O(n^2)$, sedangkan Thomas hanya memerlukan waktu dan memori orde $O(n)$. Dari sisi numerik, error Thomas juga tetap kecil dan stabil pada seluruh ukuran yang diuji. Dengan demikian, rekomendasi akhirnya jelas: untuk *banded system* yang bersifat umum dan tidak mempunyai struktur khusus yang mudah dieksploitasi, Block LU masih layak dipakai; tetapi untuk *banded system* spesial seperti matriks tridiagonal pada soal ini, gunakan algoritma compact seperti Thomas.

Referensi Technical Report I

1. *LAPACK Users' Guide*. Bagian `ILAENV`. Netlib, n.d. https://www.netlib.org/lapack/lug/node131.html

2. *LAPACK Users' Guide*. Bagian factorization untuk sistem linear. Netlib, n.d. https://www.netlib.org/lapack/lug/node68.html

3. *LAPACK DGETRF Source*. Netlib, n.d. https://www.netlib.org/lapack/explore-html/d3/d6a/dgetrf_8f_source.html

Technical Report II: Pemodelan Lintasan Bintang

Rangkuman

Laporan kedua membahas pemodelan lintasan bintang dari 300 titik observasi menggunakan model elips umum pada bidang Kartesius. Setelah konstanta persamaan konik dinormalisasi menjadi $F = 1$, persoalan berubah menjadi sistem linear *overdetermined* dengan lima parameter yang harus dicari melalui pendekatan *least squares*. Karena jumlah observasi jauh lebih besar daripada jumlah parameter, fokus utamanya bukan pada solusi eksak, melainkan pada pencarian parameter yang meminimumkan residual global antara model dan data.

Dalam kerangka ini, dua metode dibandingkan, yaitu **Normal Equation** dan **QR factorization with Householder reflections**. Normal Equation menarik karena mereduksi persoalan ke sistem kecil berukuran 5×5, tetapi ia mengkuadratkan *condition number*. Sebaliknya, QR Householder bekerja melalui dekomposisi ortogonal, sehingga secara numerik lebih stabil. Dengan demikian, perbandingan pada laporan ini tidak berhenti pada residual saja, tetapi juga menilai ketahanan metode terhadap error pembulatan dan perturbasi data.

Hasil perhitungan menunjukkan bahwa kedua metode menghasilkan koefisien yang praktis identik dan residual error yang sama, yaitu sekitar 2.5082. Meskipun demikian, QR Householder tetap lebih layak dipilih karena bekerja langsung dengan $\kappa(M) \approx 137.68$, sedangkan Normal Equation bergantung pada $M^T M$ dengan *condition number* sekitar 18,955.87. Dari solusi terbaik tersebut diperoleh persamaan elips

$$0.11899103x^2 - 0.11057894xy + 0.12597053y^2 - 0.16147331x - 0.94180659y + 1 = 0,$$

dengan pusat sekitar $(3.0343, 5.0700)$, semi-major axis 4.9331, semi-minor axis 3.0294, sudut rotasi sekitar $43.19°$, dan eksentrisitas 0.7892.

Kesimpulan utamanya adalah bahwa pada data ini kedua metode sama-sama mampu menghasilkan fit yang baik, tetapi QR Householder merupakan pilihan yang lebih defensible secara numerik. Dengan kata lain, seperti pada laporan pertama, struktur masalah tetap menentukan metode yang paling masuk akal: pada persoalan geometri least squares, kestabilan numerik harus diperlakukan sama pentingnya dengan kecocokan model terhadap data.

7 Pendahuluan

Dalam astronomi dan mekanika orbital, lintasan benda langit sering dimodelkan sebagai kurva elips. Secara konseptual, persoalan ini tampak geometris; tetapi setelah titik-titik observasi ditulis sebagai pasangan koordinat (x_i, y_i), masalah tersebut berubah menjadi pencarian koefisien persamaan konik yang paling sesuai dengan data. Karena jumlah titik observasi jauh melebihi jumlah parameter yang hendak dicari, sistem yang terbentuk tidak lagi dapat diharapkan mempunyai solusi eksak. Oleh sebab itu, formulasi yang wajar adalah *least squares*, yaitu mencari parameter yang meminimumkan residual global antara model dan data.

Studi ini membandingkan dua pendekatan standar untuk menyelesaikan persoalan tersebut. Pendekatan pertama adalah *Normal Equation*, yang sederhana karena mengubah persoalan *least squares* menjadi sistem linear kecil. Pendekatan kedua adalah *QR factorization with Householder reflections*, yang lebih panjang secara prosedural tetapi secara klasik lebih kuat dari sisi stabilitas numerik. Fokus bagian ini bukan hanya menunjukkan bagaimana kedua metode menghasilkan solusi, tetapi juga menjelaskan mengapa stabilitas numerik tetap menjadi kriteria utama walaupun residual akhir keduanya hampir sama.

8 Pembahasan

8.1 Pembentukan Model Least Squares

Persamaan elips umum pada bidang Kartesius ditulis sebagai

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0.$$

Dengan normalisasi $F = 1$, koefisien yang perlu dicari adalah

$$\theta = [A,\ B,\ C,\ D,\ E]^T.$$

Untuk setiap titik observasi (x_i, y_i), substitusi ke persamaan elips menghasilkan

$$Ax_i^2 + Bx_iy_i + Cy_i^2 + Dx_i + Ey_i = -1.$$

Jika seluruh 300 titik disusun sekaligus, maka diperoleh sistem *overdetermined*

$$M\theta \approx b,$$

dengan setiap baris matriks desain M berbentuk

$$[x_i^2,\ x_iy_i,\ y_i^2,\ x_i,\ y_i],$$

dan setiap entri ruas kanan b bernilai -1. Karena $m = 300$ dan $n = 5$, sistem ini tidak diharapkan mempunyai solusi eksak. Yang dicari adalah parameter θ yang meminimumkan $\|M\theta - b\|_2$.

Table 8.1: Struktur model least squares untuk pemodelan lintasan bintang.

Komponen	Ukuran	Peran
Matriks desain M	300×5	menyimpan fitur x_i^2, x_iy_i, y_i^2, x_i, y_i
Vektor parameter θ	5×1	koefisien elips yang dicari
Vektor ruas kanan b	300×1	seluruhnya bernilai -1

Tabel 8.1 memperlihatkan bahwa masalah ini kecil dalam dimensi parameter, tetapi kaya dalam jumlah data. Struktur seperti ini khas pada *least squares*: beban komputasi utama terletak pada bagaimana data diproyeksikan ke ruang parameter secara stabil, bukan pada ukuran sistem inti yang harus diselesaikan.

8.2 Penyelesaian dengan Normal Equation

Normal Equation diperoleh dengan meminimumkan fungsi objektif

$$\phi(\theta) = \|M\theta - b\|_2^2.$$

Dengan menurunkan $\phi(\theta)$ terhadap θ dan menyamakan gradiennya dengan nol, diperoleh syarat stasioner

$$M^T M\theta = M^T b.$$

Keuntungan langsung pendekatan ini adalah bahwa sistem yang harus diselesaikan hanya berukuran 5×5. Selain itu, matriks $M^T M$ bersifat simetris dan pada kasus ini positif definit, sehingga penyelesaiannya unik dan dapat diperoleh dengan teknik standar untuk matriks kecil (Burden dan Faires, 2011; Golub dan Van Loan, 2013).

Namun keuntungan ini disertai risiko yang penting. Matriks $M^T M$ secara umum lebih buruk kondisioning-nya daripada M sendiri. Pada data ini, $\kappa(M^T M) \approx 18{,}955.87$. Nilai tersebut menunjukkan bahwa error pembulatan atau perturbasi kecil pada data dapat diperkuat secara lebih signifikan daripada bila persoalan diselesaikan tanpa membentuk matriks normal (Golub dan Van Loan, 2013; Trefethen dan Bau, 1997).

Table 8.2: Solusi parameter menggunakan Normal Equation.

Koefisien	Nilai
A	0.11899103
B	-0.11057894
C	0.12597053
D	-0.16147331
E	-0.94180659
F	1
Residual error $\|M\theta - b\|_2$	2.5082139462

Tabel 8.2 memperlihatkan bahwa Normal Equation menghasilkan residual sebesar 2.5082139462. Secara praktis, nilai ini menyatakan akumulasi deviasi antara elips hasil model terhadap 300 titik observasi yang diberikan. Jadi, pada level akurasi residual, metode ini sudah bekerja dengan baik. Persoalannya bukan pada residual, melainkan pada ketahanan solusi tersebut terhadap error numerik.

8.3 Penyelesaian dengan QR Householder

Metode kedua memfaktorkan matriks desain sebagai

$$M = QR,$$

dengan Q matriks ortogonal dan R matriks segitiga atas. Dalam implementasi Householder, dekomposisi dibangun secara kolom demi kolom. Pada kolom ke-j, diambil subvektor dari elemen diagonal ke bawah, lalu dibentuk reflektor Householder yang menolkan

seluruh elemen di bawah diagonal pada kolom tersebut. Proses ini diulang sampai seluruh matriks berubah menjadi segitiga atas (Golub dan Van Loan, 2013; Trefethen dan Bau, 1997).

Secara konseptual, algoritmanya berjalan sebagai berikut: ambil vektor kolom yang hendak direduksi, bentuk vektor Householder u, definisikan reflektor $H_j = I - 2uu^T$, lalu aplikasikan reflektor tersebut ke submatriks yang masih aktif. Setelah semua reflektor diterapkan, diperoleh dekomposisi $M = QR$. Solusi *least squares* kemudian didapat dari sistem segitiga atas

$$R_1\theta = Q_1^T b,$$

dengan Q_1 dan R_1 menyatakan *thin QR* yang relevan terhadap jumlah parameter (Golub dan Van Loan, 2013).

```
procedure HOUSEHOLDER_QR(A)
    if rows(A) <= 1 or cols(A) = 0:
        return A

    x <- kolom pertama A
    u <- HOUSEHOLDER_VECTOR(x)
    H <- I - 2uu^T
    A <- H A

    A[2:m, 2:n] <- HOUSEHOLDER_QR(A[2:m, 2:n])
    return A
end procedure
```

Kode 8.1: Pseudocode singkat QR Householder.

Kode 8.1 ditulis secara rekursif agar ide reduksi pada submatriks aktif lebih mudah dibaca. Implementasi programnya sendiri tetap dilakukan secara iteratif, kolom demi kolom, sebagaimana yang lazim pada Householder QR.

Pada laporan sumber, akurasi dekomposisi diverifikasi melalui

$$\|M - QR\|_F \approx 9.96 \times 10^{-13}, \qquad \|Q^T Q - I\|_F \approx 1.89 \times 10^{-15},$$

yang berarti dekomposisi dan ortogonalitas Q terjaga dengan baik.

Table 8.3: Solusi parameter menggunakan QR factorization with Householder reflections.

Koefisien	Nilai
A	0.11899103
B	-0.11057894
C	0.12597053
D	-0.16147331
E	-0.94180659
F	1
Residual error $\|M\theta - b\|_2$	2.5082139462

Tabel 8.3 menunjukkan bahwa QR Householder menghasilkan koefisien dan residual yang sama dengan Normal Equation hingga presisi numerik tinggi. Dalam laporan sumber,

perbedaan residual kedua metode bahkan hanya berada pada orde $\approx 8.88 \times 10^{-16}$, yaitu setingkat *machine epsilon*. Jadi, dari sudut pandang hasil numerik mentah, kedua metode tampak setara. Di sinilah evaluasi stabilitas menjadi penting.

8.4 Perbandingan Kompleksitas FLOPs dan Memori

Untuk ukuran data $m = 300$ dan $n = 5$, biaya aritmetika kedua metode tidak berbeda jauh. Pada Normal Equation, kerja utama berasal dari pembentukan $M^T M$ dan $M^T b$, lalu penyelesaian sistem kecil. Pada QR Householder, biaya utama muncul saat membangun reflektor dan menerapkannya ke matriks desain, lalu menyelesaikan sistem segitiga atas.

Untuk $n = 5$, perbedaan FLOPs kedua metode memang kecil, sehingga pada data ini tidak wajar mengharapkan selisih runtime yang dramatis. Justru poin pentingnya adalah bahwa QR Householder mempertahankan stabilitas tanpa menambah biaya aritmetika secara berarti.

Dari sisi memori, Normal Equation perlu menyimpan $M^T M$ dan $M^T b$, sedangkan QR Householder dapat bekerja langsung pada matriks desain dengan reflektor yang disimpan implisit. Pada ukuran kecil ini selisih okupasinya belum besar, tetapi struktur in-place QR tetap lebih baik bila skala masalah diperbesar (Golub dan Van Loan, 2013; Trefethen dan Bau, 1997).

Table 8.4: Perbandingan FLOPs dan okupasi memori antara Normal Equation dan QR Householder.

Aspek	Normal Equation	QR Householder	Catatan
Langkah utama FLOPs	$2mn^2 = 15{,}000$ untuk $M^T M$	$2mn^2 - \frac{2}{3}n^3 \approx 14{,}917$	biaya dominan pembentukan faktor
Langkah lanjutan FLOPs	$2mn = 3{,}000$ untuk $M^T b$	$2mn = 3{,}000$ untuk $Q^T b$	ruas kanan diproyeksikan satu kali
Solve akhir	Cholesky $\approx \frac{1}{3}n^3 \approx 41$	back substitution $\approx n^2 = 25$	sistem inti sangat kecil
Total FLOPs	$\approx 18{,}066$	$\approx 17{,}942$	hampir sama pada kasus ini
Asimtotik waktu	$O(mn^2 + n^3)$	$O(mn^2)$	QR lebih aman saat skala membesar
Memori praktis	$mn + n^2 + n = 1{,}530$ unit	mn in-place sampai $\approx 2mn$ bila Q eksplisit	reflektor bisa disimpan implisit

8.5 Evaluasi Stabilitas Numerik

Perbedaan mendasar antara kedua metode dapat diringkas dalam hubungan

$$\kappa(M^T M) = \kappa(M)^2.$$

Hubungan ini berarti bahwa Normal Equation bekerja pada sistem yang sensitivitasnya telah dikuadratkan. Jika $\kappa(M)$ cukup besar, maka digit presisi yang hilang akibat pembulatan juga bertambah lebih cepat. Pada data ini, $\kappa(M) \approx 137.68$, sehingga $\kappa(M^T M) \approx 18{,}955.87$, sesuai dengan kuadratnya (Golub dan Van Loan, 2013; Trefethen dan Bau, 1997).

Tabel 8.5 memperjelas mengapa QR tetap dipilih. Walaupun residual kedua metode sama, QR bekerja pada representasi yang lebih sehat dari sisi numerik. Dalam laporan sumber juga dibahas analisis perturbasi pada ruas kanan. Secara umum, perubahan relatif solusi memenuhi batas atas berbentuk

$$\frac{\|\delta\theta\|}{\|\theta\|} \lesssim \kappa(\cdot)\, \frac{\|\delta b\|}{\|b\|}.$$

Table 8.5: Perbandingan stabilitas numerik antara Normal Equation dan QR Householder.

Metrik	Normal Equation	QR Householder	Implikasi
Matriks yang diselesaikan	M^TM berukuran 5×5	R berukuran 5×5	QR tidak membentuk matriks normal
Condition number	$\kappa(M^TM) \approx 18{,}955.87$	$\kappa(M) \approx 137.68$	Normal Equation jauh lebih sensitif
Digit presisi terancam	≈ 4.28 digit	≈ 2.14 digit	QR lebih hemat presisi
Residual error	2.5082139462	2.5082139462	residual sama tidak berarti stabilitas sama

Jika yang muncul adalah $\kappa(M^TM)$, maka penguatan error potensial menjadi jauh lebih besar dibanding bila yang muncul hanya $\kappa(M)$. Pada data sekarang, perbedaannya belum menimbulkan penyimpangan solusi yang kasat mata, tetapi secara metodologis QR Householder tetap merupakan pilihan yang lebih robust.

8.6 Persamaan Elips Akhir dan Interpretasi Geometris

Berdasarkan evaluasi di atas, metode terbaik yang dipilih adalah QR Householder. Persamaan elips akhirnya adalah

$$0.11899103x^2 - 0.11057894xy + 0.12597053y^2 - 0.16147331x - 0.94180659y + 1 = 0.$$

Verifikasi awal terhadap jenis kurva dapat dilakukan melalui diskriminan kuadratik

$$B^2 - 4AC = -0.04773 < 0,$$

yang memastikan bahwa kurva tersebut memang sebuah elips. Justifikasinya singkat: bagian kuadratik dapat ditulis sebagai $q(x, y) = [x\ y]Q[x\ y]^T$ dengan

$$Q = \begin{bmatrix} A & B/2 \\ B/2 & C \end{bmatrix},$$

dan karena Q simetris, ia dapat didiagonalisasi secara ortogonal. Diskriminan memenuhi $B^2 - 4AC = -4\det(Q)$, sehingga tanda diskriminan memberi informasi langsung tentang tanda eigenvalue Q; pada kasus ini keduanya bertanda sama dan level set yang dihasilkan adalah elips (Anton dan Rorres, 2013).

Pusat elips diperoleh dari titik stasioner fungsi kuadratik, yaitu dengan menyelesaikan sistem

$$\frac{\partial F}{\partial x} = 2Ax + By + D = 0, \qquad \frac{\partial F}{\partial y} = Bx + 2Cy + E = 0.$$

Justifikasi singkatnya adalah sebagai berikut. Mulailah dari paraboloid standar

$$z = x^2 + y^2,$$

yang mempunyai titik minimum di $(0, 0)$. Untuk level set $z = 1$, kurva yang diperoleh adalah lingkaran berpusat di asal, sehingga pusat lingkaran itu tepat sama dengan titik minimum paraboloid. Elips hasil kita dapat diperoleh dari lingkaran tersebut melalui substitusi bijektif

$$\begin{bmatrix} X \\ Y \end{bmatrix} = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} + \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} ax \\ by \end{bmatrix},$$

atau setara dengan

$$X = x_0 + ax\cos\theta - by\sin\theta, \qquad Y = y_0 + ax\sin\theta + by\cos\theta.$$

Karena translasi, rotasi, dan reskalasi semuanya bijektif, pusat lingkaran pada model standar berkorespondensi satu-satu dengan pusat elips pada koordinat asli. Itulah sebabnya pusat elips tetap diperoleh dari titik stasioner $\nabla F = 0$ (Stewart, 2015).

Dari sini diperoleh pusat

$$(x_0, y_0) = (3.0343,\ 5.0700).$$

Untuk menentukan ukuran dan orientasi sumbu utama, digunakan lagi matriks kuadratik

$$Q = \begin{bmatrix} A & B/2 \\ B/2 & C \end{bmatrix},$$

dengan eigenvalue sekitar $\lambda_{\min} = 0.06708$ dan $\lambda_{\max} = 0.17788$. Setelah translasi ke pusat dan diagonal-isasi ortogonal $Q = R\Lambda R^T$, persamaan elips berubah ke bentuk standar

$$\lambda_1 u^2 + \lambda_2 v^2 = \rho,$$

dengan $\rho \approx 1.6324$. Dari sini panjang sumbu diperoleh dengan langsung mencocokkan ke rumus standar $\frac{u^2}{a^2} + \frac{v^2}{b^2} = 1$, sehingga

$$a = \sqrt{\frac{\rho}{\lambda_{\min}}}, \qquad b = \sqrt{\frac{\rho}{\lambda_{\max}}}.$$

Nilai-nilai ini menghasilkan semi-major axis $a = 4.9331$ dan semi-minor axis $b = 3.0294$. Arah eigenvector yang bersesuaian dengan $\lambda_{\min}$ memberikan sudut rotasi elips sekitar 43.19° terhadap sumbu-x positif. Justifikasinya singkat: setelah bentuk kuadratik didiagonalisasi, arah eigenvector tersebut tepat menjadi arah sumbu mayor, sehingga sudut rotasi cukup dibaca dari segitiga yang mewakili arah itu, yakni $\tan\theta = \frac{v_2}{v_1}$ (Anton dan Rorres, 2013).

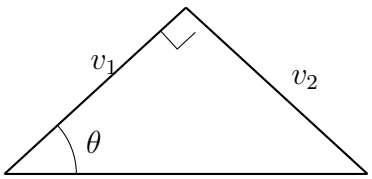

Figure 1: Segitiga bantu untuk membaca sudut rotasi dari arah sumbu mayor.

Gambar 1 hanya berfungsi sebagai aid visual: arah sumbu mayor direpresentasikan oleh segitiga siku-siku sederhana, sehingga θ langsung dapat dibaca dari perbandingan dua sisi yang ditandai. Dari rasio kedua sumbu tersebut juga diperoleh eksentrisitas $e = 0.7892$.

Tabel 8.6 menegaskan bahwa koefisien aljabar yang diperoleh benar-benar memiliki arti geometris yang jelas. Elips hasil model tidak sejajar dengan sumbu koordinat, cukup lonjong, dan memiliki pusat yang bergeser dari titik asal. Dengan demikian, solusi least squares tidak hanya kecil residual-nya, tetapi juga menghasilkan bentuk kurva yang secara geometris masuk akal untuk mewakili lintasan orbit yang diamati.

Table 8.6: Karakteristik geometris elips hasil pemodelan lintasan bintang.

Parameter	Nilai
Pusat (x_0, y_0)	$(3.0343,\ 5.0700)$
Semi-major axis a	4.9331
Semi-minor axis b	3.0294
Sudut rotasi θ	$43.19°$
Eksentrisitas e	0.7892
Diskriminan $B^2 - 4AC$	-0.04773

8.7 Validasi Visual

Laporan sumber juga menyertakan dua visualisasi penting. Visualisasi pertama menumpangkan kurva elips hasil QR Householder di atas 300 titik observasi. Hasil overlay tersebut menunjukkan bahwa kurva mengikuti sebaran data dengan baik dan tidak menyisakan deviasi sistematis yang mencolok. Visualisasi kedua membandingkan kurva hasil Normal Equation dan QR Householder dalam satu bidang yang sama. Kedua kurva secara visual nyaris tidak dapat dibedakan, konsisten dengan fakta bahwa residual keduanya identik dan selisih parameter hanya muncul pada orde sangat kecil.

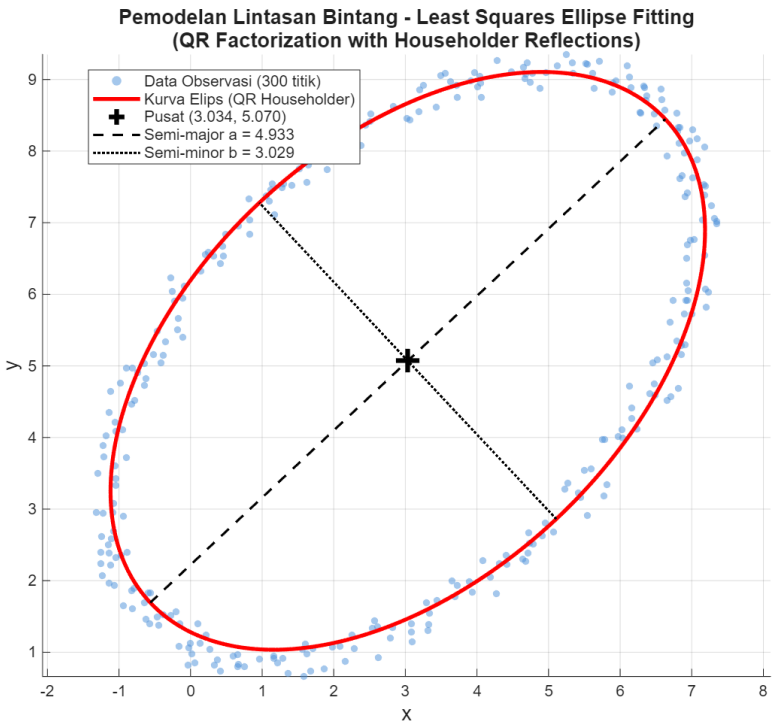

Figure 2: Overlay data observasi dengan elips hasil QR Householder.

Gambar 2 mendukung interpretasi bahwa model hasil QR Householder memang menangkap bentuk global lintasan dengan baik. Sumbu mayor, sumbu minor, dan pusat elips yang dihitung secara analitik juga konsisten dengan visualisasi ini.

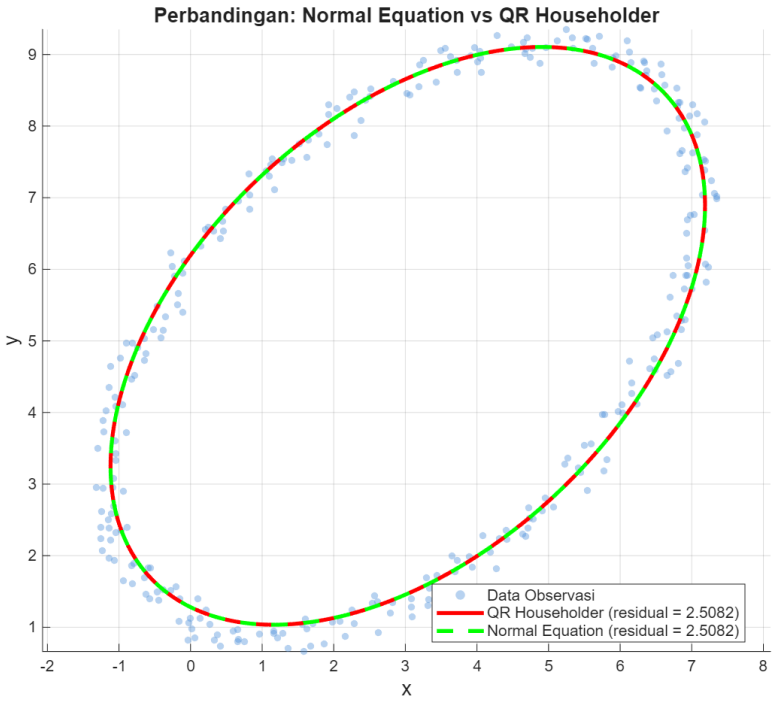

Figure 3: Perbandingan visual kurva Normal Equation dan QR Householder.

Gambar 3 memperlihatkan bahwa kedua kurva praktis bertumpuk. Dengan kata lain, bila yang dilihat hanya hasil akhir pada data ini, keduanya memang tampak sama baik. Ini menjelaskan mengapa residual keduanya sama, sekaligus menegaskan bahwa alasan pemilihan QR bukan karena ia memberi fit yang secara visual jauh berbeda, melainkan karena ia mencapainya dengan stabilitas numerik yang lebih baik.

Dengan demikian, validasi visual mendukung dua kesimpulan numerik sekaligus. Pertama, model elips yang diperoleh memang cukup baik untuk menangkap pola lintasan bintang. Kedua, perbedaan antara dua metode bukan terletak pada kualitas fit akhir yang terlihat oleh mata, melainkan pada stabilitas cara solusi itu diperoleh.

9 Kesimpulan

Pemodelan lintasan bintang pada bagian ini memperlihatkan bagaimana persoalan geometri dapat ditangani secara sistematis melalui aljabar linear numerik. Dengan memodelkan 300 titik observasi sebagai sistem linear *overdetermined*, dua metode least squares dapat dibandingkan secara langsung. Normal Equation dan QR Householder sama-sama menghasilkan koefisien elips yang nyaris identik serta residual error sebesar 2.5082139462.

Namun kesimpulan metodologisnya lebih tajam daripada itu. QR Householder tetap lebih layak dipilih karena bekerja dengan transformasi ortogonal dan tidak mengkuadratkan *condition number*. Stabilitas inilah yang membuatnya lebih dapat diandalkan bila data menjadi lebih sensitif atau bila ukuran masalah diperbesar. Persamaan elips akhir yang diperoleh juga mempunyai interpretasi geometris yang jelas: pusat di sekitar $(3.0343, 5.0700)$, semi-major axis 4.9331, semi-minor axis 3.0294, sudut rotasi $43.19°$, dan eksentrisitas

0.7892. Dengan demikian, seperti pada laporan pertama, pilihan metode yang baik pada akhirnya tetap ditentukan oleh struktur numerik masalah, bukan hanya oleh bentuk solusi akhirnya.

Referensi Technical Report II

1. Burden, R. L., dan Faires, J. D. (2011). *Numerical Analysis*. Brooks/Cole.
2. Golub, G. H., dan Van Loan, C. F. (2013). *Matrix Computations*. Johns Hopkins University Press.
3. Trefethen, L. N., dan Bau, D. (1997). *Numerical Linear Algebra*. SIAM.
4. Anton, H., dan Rorres, C. (2013). *Elementary Linear Algebra: Applications Version*. Wiley.
5. Stewart, J. (2015). *Calculus: Early Transcendentals*. Cengage.

Lampiran: Repositori dan Asal Berkas

Seluruh kode yang relevan terhadap *Technical Report I* tersedia pada repositori GitHub berikut:

https://github.com/dinovaslate/untitled1

Berkas utama yang dipakai pada laporan pertama adalah sebagai berikut.

1. `block_lu_cpu.py`, yang memuat implementasi Block LU pada CPU.
2. `block_lu_gpu.py`, yang memuat implementasi Block LU pada GPU.
3. `thomas_algorithm.py`, yang memuat implementasi Thomas algorithm.
4. `benchmark_tridiagonal_html.py`, yang digunakan untuk eksperimen perbandingan pada matriks tridiagonal.

Untuk *Technical Report II*, source program yang diberikan pengguna melalui archive nomor 2 sekarang juga tersedia dan artefak utamanya disalin ke folder `assets_nomor2`. Berkas yang paling relevan adalah sebagai berikut.

1. `pemodelan_lintasan_bintang.m`, yaitu script MATLAB utama untuk formulasi model, penyelesaian dengan Normal Equation dan QR Householder, analisis kompleksitas, serta ekstraksi parameter geometris elips.
2. `data.csv`, yaitu data 300 titik observasi yang dipakai sebagai masukan.
3. `plot_ellipse_fit.png` dan `plot_comparison.png`, yaitu visualisasi hasil yang dipakai kembali pada bagian validasi visual laporan kedua.